山东大学 计算机科学与技术 学院

数据结构与算法 课程实验报告

学号：202200130077	姓名：薛润泽	班级：人工智能
实验题目：实验 13 最小生成树		
实验学时：2	实验日期：2023.12.18	

实验目的：

使用 Prim 算法和 Kruskal 算法生成最小生成树。

软件开发工具：

DEV C++

1. 实验内容

使用 Prim 算法和 Kruskal 算法生成最小生成树。

2. 数据结构与算法描述 （整体思路描述，所需要的数据结构与算法）

所需数据结构与算法：邻接链表，最小堆，Prim 算法，Kruskal 算法

整体思路描述：

（1）建图使用邻接链表（Prim 算法中）和数组模拟的邻接链表（Kruskal 算法中）存储图，并在 graph 类的构造函数中完成图的输入。

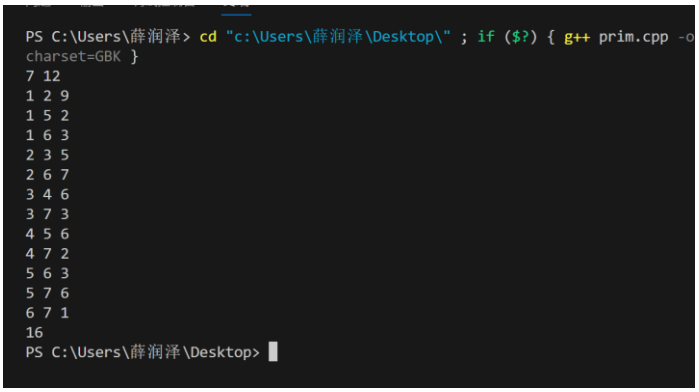
A：Prim 算法

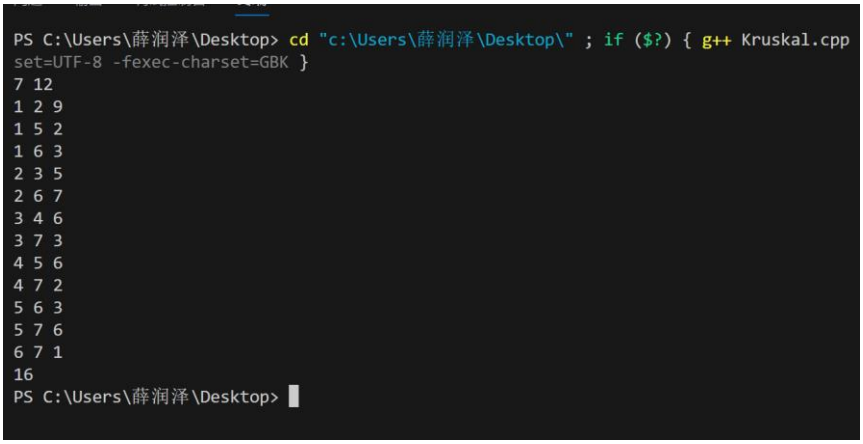
（2）new 一个 bool 类型的 sym 数组，用来标志遍历到的点是否曾经被遍历过。因为遍历到的点如果曾经被遍历到过，说明生成树中已经有一条边连接该点，且遍历出发时的点也与该点相连接，如果再连接一条边到该遍历点，则会形成环。使用最小堆来找到每个点所连接所有边中的最小边。如果最小堆中 top 元素的点没有被遍历到过，则将其权重加入答案中。选择从 1 号点开始，并将其压入最小堆中。从堆中取出堆顶元素，如果该点边没有被访问过，则将其边权重加入到答案中，并且将该点连接的所有点边压入到最小堆中。并将对应的 sym 数组中的值置为 0，代表该点被访问过。持续该循环直到最小堆为空。

B：Kruskal 算法

（2）将图中的所有点边压入最小堆中，使用并查集判断是否在答案中加入该边的权重。如果该点边的两个点都在同一个集合中，则不加入；如果两个点在两个不同的集合中，则加入该边权重，并且将两个点合并到同一个集合中。

3. 测试结果（测试输入，测试输出）





4. 分析与探讨（结果分析，若存在问题，探讨解决问题的途径）

结果与预期相同

5. 附录：实现源代码（本实验的全部源程序代码，程序风格清晰易理解，有充分的注释）

A：Prim

```
#include<iostream>

struct Node{
	Node(){
		k = 0;
		v = 0;
		next = nullptr;
	}
	Node(int key, int value){
		k = key;
		v = value;
		next = nullptr;
	}
	bool operator >(const Node& temp)const{//重载运算符
		return this->v > temp.v;
	}
	int k;
	int v;
	Node* next;
};
//优先队列
template<typename T>
class MinHeap{//最小堆类
public:
	MinHeap(){//最小堆的初始化函数，完成堆的初始化操作
		heap = new T[500002];//因为第一个 root 点在 1 处
		heapSize = 0;
	}
	~MinHeap(){
		delete []heap;
```

```
    }
     bool empty(){
          return heapSize == 0;
     }
    void push(T cnt){//插入操作
        heapSize++;
        int cntNodeloc = heapSize;
        while(cntNodeloc!=1 && heap[cntNodeloc / 2] > cnt){//没有到堆顶且其父节点值比 cnt
大，则父节点下沉, cntNodeloc 更新为父节点索引
            heap[cntNodeloc] = heap[cntNodeloc / 2];
            cntNodeloc = cntNodeloc / 2;
        }
        heap[cntNodeloc] = cnt;
    }
    void pop(){//弹出删除操作
        T lastElement =    heap[heapSize];
        heapSize --;
        int cntNodeloc = 1; //记录父索引 ,
        int child = 2;//child 存储较大的孩子的索引
        while(child <= heapSize){//child 索引没有到堆底时
            if(child    < heapSize && heap[child] > heap[child + 1]) child++;//找到左右孩子中
较小者
            if(heap[child]>lastElement) break;
            heap[cntNodeloc] = heap[child];
            cntNodeloc = child;
            child = 2 * child;
        }
        heap[cntNodeloc] = lastElement;
    }
    void Outheap(){
        for(int i = 1; i <= heapSize; i++){
            std::cout<<heap[i]<<' ';
        }
    }
    T& top(){
        return heap[1];
    }
private:
    T *heap;
    int heapSize;
};

//优先队列结束
class graph{
public:
     void push(int a, int b,int w){ //第 a 号节点与第 b 号节点之间建立路径，路径权重为 w，
```

没有特殊判定重复路

```
        Node* temp =    new Node(b, w);
        keep[a]->next = temp;
        keep[a] = temp;
    }
    graph(){
        std::cin>>n>>e;
        alist = new Node[n + 1];
        keep = new Node*[n + 1];//记录最后非空节点
        for(int i = 1; i<=n; i++){
            keep[i] = &alist[i];
        }
        for(int i = 1; i <=n ; i++) alist[i].k = i;
        int a, b, w;
        for(int i = 1; i <= e; i++){
            std::cin>>a>>b>>w;
            if(a == b)continue;//自环则跳过建图
            push(a, b, w);
            push(b, a, w);
        }
    }
    void prim(){
        bool *sym = new bool[n + 1];//用来判断点是否已经被遍历过
        for(int i = 1; i <= n; i++) sym[i] = 1;//1 代表未访问，0 代表已访问
        long long ans = 0;
        MinHeap<Node> q;
        //用 1 号点开始
        q.push(alist[1]);
        while(!q.empty()){//最小堆不空则持续循环
            Node temp = q.top();
            q.pop();
            long long rank = temp.k;
            if(sym[rank] == 1){//如果这个点没有被访问过，将其所连边压入优先队列
                ans += temp.v;
                Node* visit = (&alist[rank])->next;
                while(visit != nullptr){//将邻接链表中的所有未访问点的边压入到最小堆
中
                    if(sym[visit->k] == 1) q.push(*visit);
                    visit = visit->next;
                }
            }
            sym[rank] = 0;
        }
        std::cout<<ans<<'\n';
        delete []sym;
    }
```

```
private:
    Node* alist;
    long long n;//点的个数
    long long e;//边的条数
    Node**keep;
};

int main(){
    graph g;
    g.prim();
    return 0 ;
}
```

B：Kruskal

```
#define ll long long
#include<iostream>
template<typename T>
class MinHeap{//最小堆类
public:
    MinHeap(){//最小堆的初始化函数，完成堆的初始化操作
        heap = new T[500002];//因为第一个 root 点在 1 处
        heapSize = 0;
    }
    ~MinHeap(){
        delete []heap;
    }
     bool empty(){
          return heapSize == 0;
     }
    void push(T cnt){//插入操作
        heapSize++;
        int cntNodeloc = heapSize;
        while(cntNodeloc!=1 && heap[cntNodeloc / 2] > cnt){//没有到堆顶且其父节点值比 cnt
大，则父节点下沉, cntNodeloc 更新为父节点索引
            heap[cntNodeloc] = heap[cntNodeloc / 2];
            cntNodeloc = cntNodeloc / 2;
        }
        heap[cntNodeloc] = cnt;
    }
    void pop(){//弹出删除操作
        T lastElement =   heap[heapSize];
        heapSize --;
        int cntNodeloc = 1; //记录父索引 ,
```

```
        int child = 2;//child 存储较大的孩子的索引
        while(child <= heapSize){//child 索引没有到堆底时
            if(child   < heapSize && heap[child] > heap[child + 1]) child++;//找到左右孩子中较小者
            if(heap[child]>lastElement) break;
            heap[cntNodeloc] = heap[child];
            cntNodeloc = child;
            child = 2 * child;
        }
        heap[cntNodeloc] = lastElement;
    }
    void Outheap(){
        for(int i = 1; i <= heapSize; i++){
            std::cout<<heap[i]<<' ';
        }
    }
    T& top(){
        return heap[1];
    }
private:
    T *heap;
    int heapSize;
};
struct Edge{
    Edge(){
        To = 0;
        w = 0;
        next = 0;
    }
    ll To;//To 代表为现在的 i 点
    ll w;//代表边权重
    ll next;//To 为要去的的 j 点
    bool operator >(const Edge&temp)const{
        return this->w > temp.w;
    }
};
class graph{
public:
    void push(ll tempnow, ll tempnext, ll tempw){//push 函数，用来建图时插入点边
        cnt++;
        edge[cnt].To = tempnow;
        edge[cnt].w = tempw;
        edge[cnt].next = tempnext;
    }
    graph(){
        cnt = 0;
```

```
        std::cin>>n>>m;
        parent = new ll[n + 1];
        for(long long i = 1; i <= n; i++)parent[i] = i;//初始化并查集所用 parent 数组
        edge = new Edge[2*m + 10];
        ll cnt = 0;
        for(int ll i =    1; i <= m; i++){
            ll tempnow,tempnext, tempw;
            std::cin>>tempnow>>tempnext>>tempw;
            push(tempnow, tempnext, tempw);//建图
        }
    }
    int findacs(int son){//找到祖宗节点的同时更新祖宗节点
        if(son == parent[son]) return parent[son];
        parent[son] = findacs(parent[son]);
        return parent[son];
    }
    void merge(int a, int b){
        int acsa = findacs(a);
        int acsb = findacs(b);
        if(acsa == acsb) return;
        else if(acsa > acsb) parent[acsa] = acsb;//最后要多更新一次
        else parent[acsb] = acsa;//最后要多更新一次
        findacs(a);findacs(b);
    }
    void kruscal(){
        MinHeap<Edge> q;
        long long ans = 0;
        for(ll i = 1; i <= m; i++){
            q.push(edge[i]);
        }
        long long count = 0;
        while(!q.empty() && count < n-1){//当 q 不为空且生成树中边数小于 n - 1 时持续循
环
            Edge temp = q.top();
            q.pop();
            if(temp.To == temp.next) continue;//如果边的两个点相等，也就是自环，则直接
跳过
            if(findacs(parent[temp.To]) != findacs(parent[temp.next])){//如果该边的两个点
所在集合不是同一个集合，则将边权值加入到答案中
                count++;
                ans = ans + temp.w;
                merge(temp.To, temp.next);//合并两个集合
            }
        }
        std::cout<<ans<<'\n';
    }
```

```
private:
ll n, m;
Edge* edge;
ll cnt;
ll *parent;
};
int main(){
	graph g;
	g.kruscal();
	return 0 ;
}
```